PROGETTO METODOLOGIE DI PROGRAMAZIONE

Studente: Andrea Filipponi

Matricola: 7137945

Anno Accademico: 2024/25

Versione Eclipse: 2025-12

Sommario

1. Descrizione del Sistema Implementato 2

2. Pattern 3

2.1. Pattern Utilizzati 3

2.2. Classi coinvolte 3

2.2.1. State Pattern: Gestione del Ciclo di Vita 3

2.2.2. Static Factory Methods: Accesso agli Stati 3

2.2.3. Observer: Reattività e Disaccoppiamento 4

2.2.4. Visitor: Operazioni su Gerarchie Eterogenee 4

3. Scelte di Design e Motivazione dei Pattern 5

3.1. Gestione delle Risorse e Costi 5

3.2. Treno (Train) 5

3.3. Stato del Treno 5

3.4. Controllo del Traffico e Logging 5

4. Diagramma UML 6

1. Descrizione del Sistema Implementato

Il progetto implementa un sistema software modulare per la gestione operativa di una stazione ferroviaria. Il sistema è progettato per simulare e controllare il ciclo di vita dei treni all'interno dell'area di competenza della stazione, garantendo il rispetto dei vincoli di sicurezza e delle risorse disponibili.

Le principali funzionalità offerte sono:

Controllo del Traffico (Station): La stazione agisce come autorità centrale che autorizza le transizioni di stato dei treni (arrivo, sosta, partenza). Nessun treno può cambiare stato senza l'esplicito consenso della stazione.

Gestione Eterogenea dei Convogli: Il sistema supporta diverse tipologie di treni (**PassengerTrain** e **FreightTrain**), ciascuno con caratteristiche specifiche (numero di posti vs capacità di carico).

Monitoraggio in Tempo Reale: Ogni cambiamento di stato dei treni viene tracciato e notificato automaticamente ai componenti interessati (ad esempio **TrainLogger**).

Gestione delle Risorse (Staff Management): Prima di autorizzare la partenza di un treno, il sistema verifica automaticamente la disponibilità del personale necessario. Se le risorse sono insufficienti, il treno viene bloccato in sicurezza.

Stima dei Costi Operativi: Calcolo dinamico dei costi di gestione basato sulle caratteristiche specifiche dei treni presenti in stazione.

2. Pattern

2.1. Pattern Utilizzati

Per garantire estensibilità, manutenibilità e disaccoppiamento, sono stati applicati i seguenti Design Pattern:

- **State**
- **Static factory Methods**
- **Observer**
- **Visitor**

2.2. Classi coinvolte

2.2.1. State Pattern: Gestione del Ciclo di Vita

La classe **Train** rappresenta l'entità principale del dominio. Un aspetto critico è la gestione del suo ciclo di vita: un treno ha comportamenti che dipendono strettamente dal suo stato corrente (es. un treno in arrivo non può ripartire prima di essersi fermato).

Per evitare l'uso di flag booleani o costrutti switch-case, che avrebbero reso il codice fragile e difficile da manutenere, si è scelto di delegare la logica delle transizioni a un oggetto stato, implementando il pattern **State**. Questo è un pattern comportamentale che permette a un oggetto di alterare il proprio comportamento quando il suo stato interno cambia, incapsulando le logiche di stato in classi separate.

La classe astratta **TrainState** definisce il contratto per le operazioni dipendenti dallo stato (*stop()*, *leave()*). Le implementazioni concrete (**InArrival**, **Stopped**, **Departed**) incapsulano la logica specifica:

- Validazione delle Transizioni: Ogni stato conosce i suoi successori validi. Tentare una transizione illegale (es. da InArrival direttamente a Departed) solleva una **IllegalStateTransitionException**.
- Immutabilità: Gli stati sono oggetti immutabili.

2.2.2. Static Factory Methods: Accesso agli Stati

Per la rappresentazione degli stati concreti si è deciso di nascondere le implementazioni specifiche. Le classi estese di **TrainState** sono definite come classi private (o anonime) e non sono accessibili direttamente dai client.

L'unico punto di accesso sono i metodi esposti dalla classe astratta: **TrainState.ofInArrival()**, **TrainState.ofStopped()**, **TrainState.ofDeparted()**.

2.2.3. Observer: Reattività e Disaccoppiamento

La stazione (Controllo Traffico) e i sistemi di logging devono reagire tempestivamente ai cambiamenti di stato dei treni senza che la classe **Train** debba conoscere i dettagli di chi la sta osservando.

Si è implementato il pattern Observer tramite l'interfaccia **TrainObserver**.

Ogni volta che un treno completa una transizione di stato con successo, notifica tutti gli osservatori registrati.

Nel progetto sono state realizzate due implementazioni significative:

- **TrainLogger**: Si occupa di tracciare la storia del treno su output (o file), disaccoppiando la logica di I/O dalla logica di business.
- **StaffCheckObserver**: Un osservatore "attivo" che, al momento della transizione verso la partenza, verifica automaticamente la disponibilità delle risorse umane, bloccando il treno o lanciando allarmi in caso di carenza di personale.

2.2.4. Visitor: Operazioni su Gerarchie Eterogenee

Il sistema deve gestire operazioni complesse che variano in base al tipo concreto di treno (**PassengerTrain** vs **FreightTrain**), come il calcolo dei costi operativi o la pianificazione dello staff.

Si è applicato il pattern Visitor. Le classi dei treni espongono un metodo *accept(TrainVisitor)*, che implementa il meccanismo del Double Dispatch. La logica di business è spostata in visitor dedicati:

- **CostEstimator**: Calcola i costi basandosi su tariffe diverse per passeggeri o tonnellate di carico.
- **StaffPlanner**: Determina il numero di controllori o operai necessari.

3. Scelte di Design e Motivazione dei Pattern

3.1. Gestione delle Risorse e Costi

È sembrato fondamentale fornire funzionalità per stimare i costi operativi e il fabbisogno di personale della stazione. Per fare ciò, invece di appesantire le classi dei treni con logica di calcolo, si è utilizzato il pattern Visitor (vedi sezione Pattern). Questo ha permesso di implementare due visitatori distinti: **CostEstimator** e **StaffPlanner**.

Nel caso si vogliano aggiungere nuove metriche (es. consumo energetico), basterà creare una nuova classe che implementa **TrainVisitor**.

3.2. Treno (Train)

La classe **Train** rappresenta i convogli ferroviari, siano essi passeggeri o merci. Per questo si è scelto di definire una classe astratta che raccoglie le proprietà comuni (id, stato corrente).

Si è pensato di rendere necessario, oltre ai metodi getter di base, fornire i metodi operativi *stop()* e *leave()*. Questi metodi non modificano direttamente la variabile di stato, ma delegano l'operazione all'oggetto stato corrente; questo ci aiuta a evitare incongruenze (come far partire un treno che è ancora in arrivo).

Si è inoltre ritenuto opportuno l'utilizzo del pattern Observer, in modo da permettere alla stazione e ai logger di reagire ai cambiamenti. Sono perciò presenti nella classe astratta tutti i metodi di gestione degli osservatori (*addObserver()* e *notifyObservers()*).

Per quanto riguarda la distinzione tra **PassengerTrain** e **FreightTrain**, si è agito solo a livello di costruttori e di implementazione del metodo accept per il visitor.

3.3. Stato del Treno

Come detto nella sezione precedente, lo stato del treno è rappresentato da un'istanza di **TrainState**. Si è preferito evitare l'utilizzo di un enum, e piuttosto introdurre una classe astratta, per non violare l'OCP e per poter gestire comportamenti complessi.

Si è ritenuto opportuno definire le classi concrete (**InArrival**, **Stopped**, **Departed**) come statiche e private all'interno della classe base, in modo da non poterle istanziare direttamente dall'esterno. Infatti, l'unico modo per ottenerle è tramite gli Static Factory Methods **ofInArrival**, **ofStopped**, **ofDeparted**.

Ognuna di queste classi ridefinisce i metodi di transizione (*stop*, *leave*). Qui è presente la gestione dei vincoli descritti nei requisiti, tramite l'utilizzo di **IllegalStateTransitionException**.

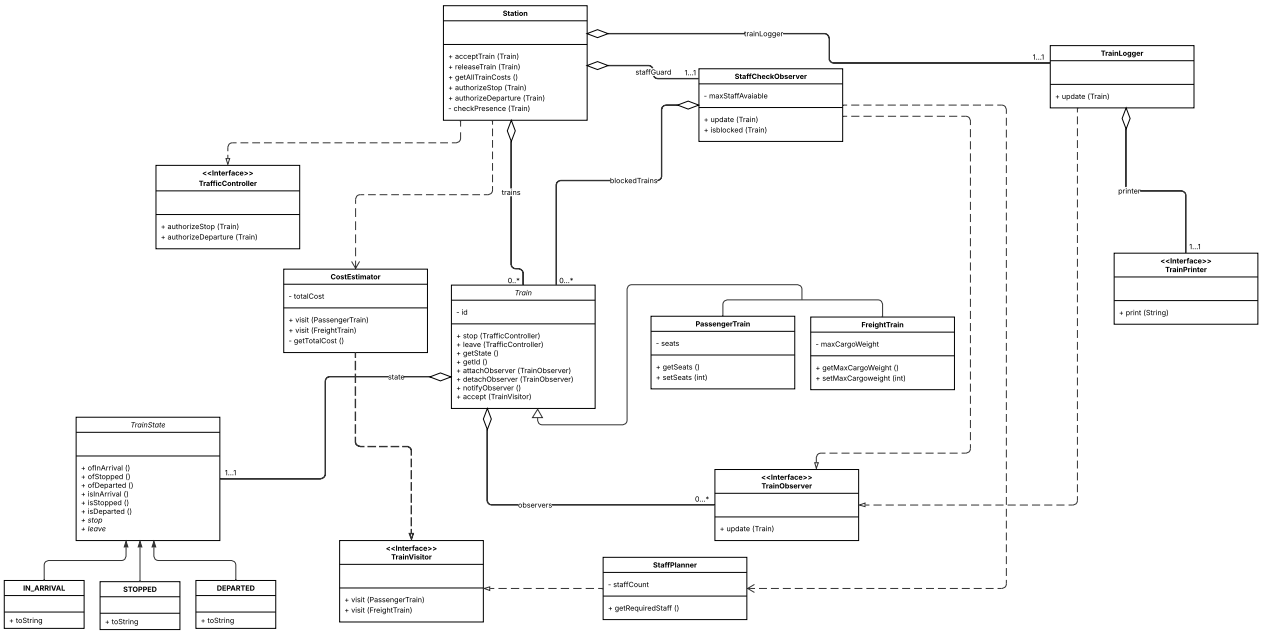
3.4. Controllo del Traffico e Logging

La gestione operativa della stazione è demandata alla classe **Station** (che implementa l'interfaccia **TrafficController**), che mantiene la lista dei treni presenti.

Si è pensato logico inserire un sistema di logging per tracciare le operazioni. Questo è stato realizzato tramite l'implementazione concreta **TrainLogger** (un observer).

Una scelta di design importante qui riguarda il testing dell'output: il logger non scrive direttamente su System.out, ma su un'interfaccia **TrainPrinter**. Nel codice di produzione si usa un'implementazione che stampa a video, mentre nei test si usa un **MockPrinter** (una classe che salva le stringhe in una lista).

4. Diagramma UML



Nel diagramma sono state omesse le classi relative alle eccezioni (es. **IllegalStateTransitionException**), che estendono **RuntimeException**, e i metodi ausiliari con visibilità package-private introdotti esclusivamente per finalità di testing. Tale scelta è stata guidata dalla volontà di massimizzare la leggibilità del diagramma, focalizzandolo sulle relazioni strutturali tra le entità principali e sull'applicazione dei Design Pattern.